

Child classes inherit all the methods of their parent classes. But a child class can override an inherited method by providing its own method with its own code. The child class's overriding method will have the same name as the parent class's method.

To illustrate this concept, let's return to the tic-tac-toe game we created in the previous chapter. This time, we'll create a new class, `MiniBoard`, that subclasses `TTTBoard` and overrides `getBoardStr()` to provide a smaller drawing of the tic-tac-toe board. The program will ask the player which board style to use. We don't need to copy and paste the rest of the `TTTBoard` methods because `MiniBoard` will inherit them.

Add the following to the end of your *tictactoe_oop.py* file to create a child class of the original `TTTBoard` class and then override the `getBoardStr()` method:

```
class MiniBoard(TTTBoard):
    def getBoardStr(self):
        """Return a tiny text-representation of the board."""
        # Change blank spaces to a '.'
        for space in ALL_SPACES:
            if self._spaces[space] == BLANK:
                self._spaces[space] = '.'

        boardStr = f'''
            {self._spaces['1']}{self._spaces['2']}{self._spaces['3']} 123
            {self._spaces['4']}{self._spaces['5']}{self._spaces['6']} 456
            {self._spaces['7']}{self._spaces['8']}{self._spaces['9']} 789'''

        # Change '.' back to blank spaces.
        for space in ALL_SPACES:
            if self._spaces[space] == '.':
                self._spaces[space] = BLANK
        return boardStr
```

As with the `getBoardStr()` method for the `TTTBoard` class, the `getBoardStr()` method for `MiniBoard` creates a multiline string of a tic-tac-toe board to display when passed to the `print()` function. But this string is much smaller, forgoing the lines between the X and O marks and using periods to indicate blank spaces.